

Landmarks (landmarks)

Luxemburg-Stadt wird durch das Tal der Alzette in zwei Teile geteilt. Lea und Tom spazieren auf jeweils einer der beiden Seiten des Tals und besuchen verschiedene Sehenswürdigkeiten. Auf Leas Seite gibt es N Sehenswürdigkeiten, und auf Toms Seite gibt es M Sehenswürdigkeiten.

Es gibt drei verschiedene Arten von Sehenswürdigkeiten:

1. eine alte Festung,
2. eine Statue oder
3. einen Brunnen.

Die Sehenswürdigkeiten auf Leas Seite sind $A_0, A_1, \dots, A_{N-1}$, und die Sehenswürdigkeiten auf Toms Seite sind $B_0, B_1, \dots, B_{M-1}$, wobei jeder Wert 1, 2 oder 3 ist.

Beide wollen die Stadt gemeinsam erkunden, wollen aber jederzeit sicherstellen, dass sie sich an einer Sehenswürdigkeit derselben Art befinden.

Betrachte ein Tupel aus vier Ganzzahlen (X_1, X_2, Y_1, Y_2) , das die beiden Wege beschreibt, die sie zurücklegen möchten, und das folgende Bedingungen erfüllt:

- $0 \leq X_1 \leq X_2 < N$ – dies steht für Leas Weg, welcher bei X_1 beginnt und bei X_2 endet,
- $0 \leq Y_1 \leq Y_2 < M$ – dies steht für Toms Weg, welcher bei Y_1 beginnt und bei Y_2 endet,
- $A_{X_1} = B_{Y_1}$ – sie starten an einer Sehenswürdigkeit derselben Art, und
- $A_{X_2} = B_{Y_2}$ – sie enden an einer Sehenswürdigkeit derselben Art.

Beide beginnen am Anfang ihrer Wege. Lea startet bei $X = X_1$, und Tom startet bei $Y = Y_1$.

In jedem Schritt wählt Lea $x \in \{-1, 0, 1\}$, und Tom wählt $y \in \{-1, 0, 1\}$, was den Gang zur vorherigen, zur nächsten Sehenswürdigkeit oder dem Verbleiben am aktuellen Ort entspricht. Anschliessend aktualisieren sie ihre Position auf $X + x$ und $Y + y$ unter der Bedingung, dass sie nach dem Schritt folgende Bedingungen erfüllen:

- $X_1 \leq X \leq X_2$ – Lea bleibt auf ihrem Weg,
- $Y_1 \leq Y \leq Y_2$ – Tom bleibt auf seinem Weg, und
- $A_X = B_Y$ – sie befinden sich an einer Sehenswürdigkeit derselben Art.

Das Tupel (X_1, X_2, Y_1, Y_2) ist gültig, wenn es möglich ist, null oder mehr Schritte anzuwenden, bis $X = X_2$ und $Y = Y_2$ gilt.

Es gibt Q Abfragen. Bei jeder Abfrage wird ein Tupel aus vier Ganzzahlen (X_1, X_2, Y_1, Y_2) angegeben, und deine Aufgabe besteht darin, festzustellen, ob das Tupel gültig ist.

Implementierung

Du musst eine einzelne .cpp-Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest du eine Vorlage `landmarks.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktionen implementieren:

C++

```
void init(vector<int> A, vector<int> B)
```

- A ist ein Array mit N Ganzzahlen.
- B ist ein Array mit M Ganzzahlen.
- Diese Funktion wird genau einmal aufgerufen, bevor `query` aufgerufen wird.

C++	<code>bool query(int X1, int X2, int Y1, int Y2)</code>
-----	---

- X_1, X_2, Y_1 und Y_2 sind Ganzzahlen, welche die folgenden Bedingungen erfüllen:
 - $0 \leq X_1 \leq X_2 < N$,
 - $0 \leq Y_1 \leq Y_2 < M$,
 - $A_{X_1} = B_{Y_1}$, und
 - $A_{X_2} = B_{Y_2}$.
- Falls das Tuple (X_1, X_2, Y_1, Y_2) gültig ist, muss die Funktion `true` zurückgeben. Ansonsten muss die Funktion `false` zurückgeben.
- Diese Funktion wird genau Q Mal aufgerufen.

Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die du verwenden kannst, um deine Lösung lokal zu testen. Der Beispielgrader liest die Eingangsdaten aus der Datei `stdin`, ruft die Funktion `init` einmal auf, ruft die Funktion `query` Q Mal auf, und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus $Q + 3$ Zeilen, die Folgendes enthalten:

- Zeile 1: Die Ganzzahlen N , M und Q .
- Zeile 2: N Ganzzahlen, $A_0, A_1, \dots, A_{N-1}$.
- Zeile 3: M Ganzzahlen, $B_0, B_1, \dots, B_{M-1}$.
- Zeile $4 + i$ ($0 \leq i < Q$): Die Ganzzahlen X_1, X_2, Y_1 und Y_2 .

Die Ausgabe besteht aus Q Zeilen, wobei die i -te Zeile den Wert beinhaltet, welcher vom der i -ten Aufruf der Funktion `query` (`true` oder `false`) beinhaltet.

Einschränkungen

- $1 \leq N, M, Q \leq 100\,000$.
- $1 \leq A_i, B_i \leq 3$ für alle i .

Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Um die Punktzahl zu erhalten, die einer Teilaufgabe zugeordnet ist, musst du alle darin enthaltenen Testfälle korrekt lösen.

- **Teilaufgabe 0 [0 Punkte]:** Beispiel-Testfälle.
- **Teilaufgabe 1 [10 Punkte]:** $N, M, Q \leq 5000$ und $A_0, A_1, \dots, A_{N-1}, B_0, B_1, \dots, B_{M-1} \leq 2$. (Es gibt nur zwei Arten von Sehenswürdigkeiten)
- **Teilaufgabe 2 [10 Punkte]:** $A_0, A_1, \dots, A_{N-1}, B_0, B_1, \dots, B_{M-1} \leq 2$. (Es gibt nur zwei Arten von Sehenswürdigkeiten)
- **Teilaufgabe 3 [10 Punkte]:** $N, M, Q \leq 5000$ und $A_i = (i \bmod 3) + 1$ für alle i .
- **Teilaufgabe 4 [15 Punkte]:** $A_i = (i \bmod 3) + 1$ für alle i .
- **Teilaufgabe 5 [10 Punkte]:** $N, M, Q \leq 300$.
- **Teilaufgabe 6 [25 Punkte]:** $N, M, Q \leq 5000$.
- **Teilaufgabe 7 [20 Punkte]:** Keine weiteren Einschränkungen.

Beispiele

stdin	stdout
5 7 4	true
1 2 3 2 1	false
1 1 2 3 2 3 1	true
0 2 0 5	false
2 4 3 6	
1 1 4 4	
1 1 2 4	

Erklärung

$N = 5$, $M = 7$, $Q = 4$, mit $A = [1, 2, 3, 2, 1]$ und $B = [1, 1, 2, 3, 2, 3, 1]$.

Abfrage 1: $(X_1, X_2, Y_1, Y_2) = (0, 2, 0, 5)$ ist gültig, da der folgende Weg möglich ist:

Schritt	1	2	3	4	5	6
X	0	0	1	2	1	2
Y	0	1	2	3	4	5
Art der Sehenswürdigkeit	1	1	2	3	2	3

Abfrage 2: $(X_1, X_2, Y_1, Y_2) = (2, 4, 3, 6)$ ist nicht gültig. Es gibt kein Weg, welcher die Bedingungen erfüllt.

Abfrage 3: $(X_1, X_2, Y_1, Y_2) = (1, 1, 4, 4)$ ist gültig.

Abfrage 4: $(X_1, X_2, Y_1, Y_2) = (1, 1, 2, 4)$ ist nicht gültig.